

Lab 7 - Data anonymity

1 Data anonymity

In this lab, we will learn how to use data anonymity techniques to protect data privacy by using Pandas.

1.1 Objectives

1. Review basic use of Pandas
2. Achieve K-anonymity
3. Achieve l-diversity
4. Achieve t-closeness

2 Basic use of Pandas

1. What is Pandas?

Pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, **real world** data analysis in Python. Additionally, it has the broader goal of becoming **the most powerful and flexible open source data analysis / manipulation tool available in any language**. It is already well on its way toward this goal.

```
[2]: pip install pandas
```

```
Collecting pandas
```

```
  Downloading pandas-1.3.5-cp37-cp37m-macosx_10_9_x86_64.whl (11.0 MB)
```

```
    |                                     | 11.0 MB 1.2 MB/s  eta 0:00:01
```

```
Collecting pytz>=2017.3
```

```
  Downloading pytz-2023.3.post1-py2.py3-none-any.whl (502 kB)
```

```
    |                                     | 502 kB 15.9 MB/s eta 0:00:01
```

```
Requirement already satisfied: numpy>=1.17.3; platform_machine !=
```

```
"aarch64" and platform_machine != "arm64" and python_version < "3.10" in
```

```
/Users/wwq/opt/anaconda3/envs/Share_improve/lib/python3.7/site-packages (from
```

```
pandas) (1.21.5)
```

```
Requirement already satisfied: python-dateutil>=2.7.3 in
```

```
/Users/wwq/opt/anaconda3/envs/Share_improve/lib/python3.7/site-packages (from
```

```
pandas) (2.8.1)
Requirement already satisfied: six>=1.5 in
/Users/wwq/opt/anaconda3/envs/Share_improve/lib/python3.7/site-packages (from
python-dateutil>=2.7.3->pandas) (1.14.0)
Installing collected packages: pytz, pandas
Successfully installed pandas-1.3.5 pytz-2023.3.post1
Note: you may need to restart the kernel to use updated packages.
```

2. Import Pandas

Importing a library means loading it into the memory and then it's there for you to work with. In order to import Pandas all you have to do is run the following code:

```
[1]: import pandas as pd # import the Pandas library
import numpy as np
```

3. Construct a dictionary

Example:

```
data = {'First Column Name': ['First value', 'Second value',...],

        'Second Column Name': ['First value', 'Second value',...],
        ....
}
```

```
[2]: data = {'Name': ['Alice', 'Betty', 'Charles', 'David', 'Emily', 'Fred'],
             'Zipcode': [47677, 47602, 47678, 47905, 47909, 47906],
             'Age': [29, 22, 27, 43, 42, 47],
             'Sex': ['F', 'F', 'M', 'M', 'F', 'M'],
             'Disease': ['Ovarian Cancer', 'Ovarian Cancer',
                        'Prostate Cancer', 'Flu', 'Heart Disease', 'Heart Disease']}
```

4. Create a dataframe from dictionary

In order to convert a certain Python object (dictionary, lists etc) the basic command is:

```
pd.DataFrame()
```

Two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns).

```
[3]: df = pd.DataFrame(data)
df
```

```
[3]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease

2.1 View and inspect data

1. Get the first n rows

```
[4]: df.head(5)
```

```
[4]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease

2. Get the last n rows

```
[5]: df.tail(3)
```

```
[5]:
```

	Name	Zipcode	Age	Sex	Disease
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease

3. Get the number of rows and columns

```
[6]: df.shape
```

```
[6]: (6, 5)
```

4. Get the index, datatype and memory information.

```
[7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column    Non-Null Count  Dtype
---  -
0   Name      6 non-null      object
1   Zipcode   6 non-null      int64
2   Age       6 non-null      int64
3   Sex       6 non-null      object
4   Disease   6 non-null      object
dtypes: int64(2), object(3)
memory usage: 368.0+ bytes
```

5. Get the mean of all columns

```
[8]: df.mean()
```

```
/Users/wwq/opt/anaconda3/envs/Share_improve/lib/python3.7/site-
packages/ipykernel_launcher.py:1: FutureWarning: Dropping of nuisance columns in
DataFrame reductions (with 'numeric_only=None') is deprecated; in a future
```

version this will raise `TypeError`. Select only valid columns before calling the reduction.

"""Entry point for launching an IPython kernel.

```
[8]: Zipcode    47779.5
     Age        35.0
     dtype: float64
```

6. Get the correlation between columns in a data frame

```
[9]: df.corr()
```

```
[9]:      Zipcode      Age
     Zipcode  1.000000  0.983538
     Age      0.983538  1.000000
```

7. Get the highest value in each column

```
[10]: df.max()
```

```
[10]: Name          Fred
     Zipcode      47909
     Age          47
     Sex          M
     Disease  Prostate Cancer
     dtype: object
```

```
[ ]:
```

2.2 Selection of Data

1. Select columns

```
[11]: df['Sex']
```

```
[11]: 0    F
     1    F
     2    M
     3    M
     4    F
     5    M
     Name: Sex, dtype: object
```

```
[12]: df[['Age', 'Sex']]
```

```
[12]:   Age Sex
     0  29  F
     1  22  F
     2  27  M
     3  43  M
```

```
4  42  F
5  47  M
```

2. Select rows by position

```
[13]: df.iloc[0]
```

```
[13]: Name          Alice
      Zipcode      47677
      Age         29
      Sex          F
      Disease  Ovarian Cancer
      Name: 0, dtype: object
```

```
[14]: df.iloc[:3]
```

```
[14]:   Name  Zipcode  Age Sex  Disease
0  Alice   47677   29  F   Ovarian Cancer
1  Betty   47602   22  F   Ovarian Cancer
2  Charles 47678   27  M  Prostate Cancer
```

Select an element

```
[15]: df.iloc[0,0]
```

```
[15]: 'Alice'
```

3. Select rows by index

```
[16]: df.loc[0]
```

```
[16]: Name          Alice
      Zipcode      47677
      Age         29
      Sex          F
      Disease  Ovarian Cancer
      Name: 0, dtype: object
```

```
[17]: df.loc[:3]
```

```
[17]:   Name  Zipcode  Age Sex  Disease
0  Alice   47677   29  F   Ovarian Cancer
1  Betty   47602   22  F   Ovarian Cancer
2  Charles 47678   27  M  Prostate Cancer
3  David   47905   43  M           Flu
```

```
[18]: df.loc[0, 'Name']
```

```
[18]: 'Alice'
```

4. Replace values with different values

Replace values given in `to_replace` with `value`. Values of the DataFrame are replaced with other values dynamically.

```
df.replace(to_replace=None, value=None, inplace=False)
```

```
[19]: df.replace('Flu', 'Lung Cancer')
```

```
[19]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Lung Cancer
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease

3 Achieve K-anonymity

K-anonymity: Each released record should be indistinguishable from at least (k-1) others on its QI attributes.

1. Import a table or construct a table

```
[20]: data = {'Name': ['Alice', 'Betty', 'Charles', 'David', 'Emily', 'Fred',  
                      'Tom', 'Alex', 'Vincent', 'Serena'],  
             'Zipcode': [47677, 47602, 47678, 47905, 47909, 47906,  
                        47433, 47489, 47422, 47443],  
             'Age': [29, 22, 27, 43, 42, 47,  
                    32, 33, 34, 38],  
             'Sex': ['F', 'F', 'M', 'M', 'F', 'M',  
                    'M', 'M', 'M', 'F'],  
             'Disease': ['Ovarian Cancer', 'Ovarian Cancer',  
                        'Prostate Cancer', 'Flu', 'Heart Disease', 'Heart Disease',  
                        'Flu', 'Flu', 'Flu', 'Heart Disease']}
```

```
[21]: df = pd.DataFrame(data)  
df
```

```
[21]:
```

	Name	Zipcode	Age	Sex	Disease
0	Alice	47677	29	F	Ovarian Cancer
1	Betty	47602	22	F	Ovarian Cancer
2	Charles	47678	27	M	Prostate Cancer
3	David	47905	43	M	Flu
4	Emily	47909	42	F	Heart Disease
5	Fred	47906	47	M	Heart Disease
6	Tom	47433	32	M	Flu
7	Alex	47489	33	M	Flu
8	Vincent	47422	34	M	Flu

9 Serena 47443 38 F Heart Disease

Goal: release a table with 6 records that achieves 3-anonymity

2. Remove key attribute

Key attributes: Name, address, phone number- uniquely identifying!

```
[22]: del df['Name']
df
```

```
[22]:
```

	Zipcode	Age	Sex	Disease
0	47677	29	F	Ovarian Cancer
1	47602	22	F	Ovarian Cancer
2	47678	27	M	Prostate Cancer
3	47905	43	M	Flu
4	47909	42	F	Heart Disease
5	47906	47	M	Heart Disease
6	47433	32	M	Flu
7	47489	33	M	Flu
8	47422	34	M	Flu
9	47443	38	F	Heart Disease

3. Obscure quasi-identifier

quasi-identifier: 5-digit ZIP code, birth date, gender, Can be used for linking anonymized dataset with other datasets.

```
[23]: # obscure zipcode to 476**
df['Zipcode'].replace([47677,47602,47678], '476**',inplace=True)
df
```

```
[23]:
```

	Zipcode	Age	Sex	Disease
0	476**	29	F	Ovarian Cancer
1	476**	22	F	Ovarian Cancer
2	476**	27	M	Prostate Cancer
3	47905	43	M	Flu
4	47909	42	F	Heart Disease
5	47906	47	M	Heart Disease
6	47433	32	M	Flu
7	47489	33	M	Flu
8	47422	34	M	Flu
9	47443	38	F	Heart Disease

```
[24]: # obscure zipcode to 479**
df['Zipcode'].replace([47905,47909,47906], '479**',inplace=True)
df
```

```
[24]:
```

	Zipcode	Age	Sex	Disease
0	476**	29	F	Ovarian Cancer

1	476**	22	F	Ovarian Cancer
2	476**	27	M	Prostate Cancer
3	479**	43	M	Flu
4	479**	42	F	Heart Disease
5	479**	47	M	Heart Disease
6	47433	32	M	Flu
7	47489	33	M	Flu
8	47422	34	M	Flu
9	47443	38	F	Heart Disease

```
[25]: # obscure zipcode to 474**
df['Zipcode'].replace([47433,47489,47422,47443], '474**',inplace=True)
df
```

```
[25]:
```

	Zipcode	Age	Sex	Disease
0	476**	29	F	Ovarian Cancer
1	476**	22	F	Ovarian Cancer
2	476**	27	M	Prostate Cancer
3	479**	43	M	Flu
4	479**	42	F	Heart Disease
5	479**	47	M	Heart Disease
6	474**	32	M	Flu
7	474**	33	M	Flu
8	474**	34	M	Flu
9	474**	38	F	Heart Disease

```
[26]: # obscure age to 2*
df['Age'].replace([29,22,27], '2*',inplace=True)
df
```

```
[26]:
```

	Zipcode	Age	Sex	Disease
0	476**	2*	F	Ovarian Cancer
1	476**	2*	F	Ovarian Cancer
2	476**	2*	M	Prostate Cancer
3	479**	43	M	Flu
4	479**	42	F	Heart Disease
5	479**	47	M	Heart Disease
6	474**	32	M	Flu
7	474**	33	M	Flu
8	474**	34	M	Flu
9	474**	38	F	Heart Disease

```
[27]: # obscure age to 4*
df['Age'].replace([43,42,47], '4*',inplace=True)
df
```

```
[27]: Zipcode Age Sex      Disease
0    476** 2*   F   Ovarian Cancer
1    476** 2*   F   Ovarian Cancer
2    476** 2*   M   Prostate Cancer
3    479** 4*   M           Flu
4    479** 4*   F   Heart Disease
5    479** 4*   M   Heart Disease
6    474** 32   M           Flu
7    474** 33   M           Flu
8    474** 34   M           Flu
9    474** 38   F   Heart Disease
```

```
[28]: # obscure age to 3*
df['Age'].replace([32,33,34,38], '3*',inplace=True)
df
```

```
[28]: Zipcode Age Sex      Disease
0    476** 2*   F   Ovarian Cancer
1    476** 2*   F   Ovarian Cancer
2    476** 2*   M   Prostate Cancer
3    479** 4*   M           Flu
4    479** 4*   F   Heart Disease
5    479** 4*   M   Heart Disease
6    474** 3*   M           Flu
7    474** 3*   M           Flu
8    474** 3*   M           Flu
9    474** 3*   F   Heart Disease
```

```
[29]: # obscure sex to *
df['Sex'].replace(['F','M'], '*',inplace=True)
df
```

```
[29]: Zipcode Age Sex      Disease
0    476** 2*   *   Ovarian Cancer
1    476** 2*   *   Ovarian Cancer
2    476** 2*   *   Prostate Cancer
3    479** 4*   *           Flu
4    479** 4*   *   Heart Disease
5    479** 4*   *   Heart Disease
6    474** 3*   *           Flu
7    474** 3*   *           Flu
8    474** 3*   *           Flu
9    474** 3*   *   Heart Disease
```

4. Release the table with 6 records that achieves 3-anonymity

```
[30]: release_df_1 = df[:6]
release_df_1
```

```
[30]:  Zipcode Age Sex      Disease
      0   476** 2*   *   Ovarian Cancer
      1   476** 2*   *   Ovarian Cancer
      2   476** 2*   *   Prostate Cancer
      3   479** 4*   *           Flu
      4   479** 4*   *   Heart Disease
      5   479** 4*   *   Heart Disease
```

4 Achieve l-diversity

l-diversity:

An equivalent class is l-diverse if contains at least l ‘well represented” values for the sensitive attribute S. A table is l-diverse if every equivalent class is l-diverse.

Goal: Release the table with 9 records that achieves 3-anonymity and 2-diversity

Based on the operation to achieve k-anonymity, we find that the first two classes satisfy 3-anonymity and 2-diversity.

Hence, now we need to choose three records from the last class that satisfy 3-anonymity and 2-diversity.

```
[31]: release_df_2 = df.drop(8)
      release_df_2
```

```
[31]:  Zipcode Age Sex      Disease
      0   476** 2*   *   Ovarian Cancer
      1   476** 2*   *   Ovarian Cancer
      2   476** 2*   *   Prostate Cancer
      3   479** 4*   *           Flu
      4   479** 4*   *   Heart Disease
      5   479** 4*   *   Heart Disease
      6   474** 3*   *           Flu
      7   474** 3*   *           Flu
      9   474** 3*   *   Heart Disease
```

5 Achieve t-closeness

What is t-closeness?

An equivalence class is said to have t-closeness if the distance between the distribution of a sensitive attribute in this class and the distribution of the attribute in the whole table is no more than a threshold t.

The distance between two distributions commonly is calculated by the wasserstein distance/earth mover’s distance (EMD).

wasserstein_distance

Wasserstein distances arise from the so-called Optimal Transport theory. It aims to provide a special way to compare probability distributions.

1. Import library of `wasserstein_distance`

```
[32]: from scipy.stats import wasserstein_distance
```

2. Express the disease with numbers

1: Ovarian Cancer

2: Prostate Cancer

3: Flu

4: Heart Disease

3. Calculate the distribution of the each class

```
[33]: p_1 = [1,1,2]
```

```
p_2 = [3,4,4]
```

```
p_3 = [3,3,4]
```

4. Calculate the distribution in the whole table

```
[34]: q = [1,1,2,3,4,4,3,3,4]
```

5. Calculate the `wasserstein_distance` between `class_1`, `class_2`, `class_3` and the whole table.

```
[35]: d_1 = wasserstein_distance(p_1,q)
```

```
print(d_1)
```

```
d_2 = wasserstein_distance(p_2,q)
```

```
print(d_2)
```

```
d_3 = wasserstein_distance(p_3,q)
```

```
print(d_3)
```

1.4444444444444446

0.8888888888888888

0.5555555555555556

6. Calculate the t-closeness

```
[36]: t = max(d_1,d_2,d_3)
```

```
print(t)
```

1.4444444444444446

6 summary

K-anonymity: Each released record should be indistinguishable from at least $(k-1)$ others on its QI attributes.

l-Diversity: Sensitive attributes must be “l-diverse” within each quasi-identifier equivalence class.

t-closeness : t-closeness requires that the distribution of a sensitive attribute in any eq. class is close to the distribution of a sensitive attribute in the overall table.

7 Quiz

1. Construct a table that includes key attributes, quasi-identifiers and sensitive attributes and the table has at least 15 rows.
2. Release the table with 10 records that achieves 3-anonymity and 2-diversity and calculate the value of t in the t-closeness.
3. Given a dataset, release the dataset that achieves 3-anonymity and 2-diversity and calculate the value of t in the t-closeness.

[]: